



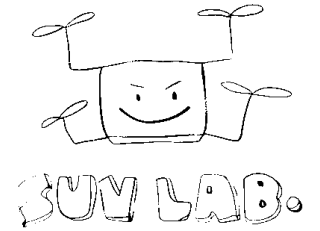
충북대학교

PYTHON 프로그래밍 · PART 2-2

NumPy

다차원 배열 연산 및 행렬 제어

충북대학교 지능로봇공학과 **문성태** 교수



What is NumPy

파이썬을 수치 계산 언어로 만든 라이브러리

Who

- **Travis Oliphant**
2005년 초판 공개
- **뿌리** — Numeric(1995),
Numarray(2001) 통합
- **지금** — NumFOCUS 가 후원하는 커
뮤니티 개발

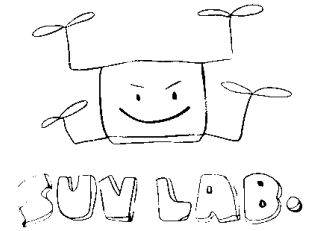
Why

- **문제**
파이썬 리스트는 크고 느리다
- **해법**
연속 메모리에 담고 연산은 C 로
- **결과**
루프 없이 배열 전체를 한 번에 계산

Where

- **기반**
SciPy, Pandas, scikit-
learn, PyTorch
- **사례**
LIGO 중력파 검출
- **라이선스**
BSD 3-Clause, 상업적 사용 자유

버전 — 1.0 은 2006년, 2.0 은 2024년. 이 강의는 2.x 기준이다.



Why NumPy

파이썬 리스트

```
1 a = [1, 2, 3]
2 a * 2
3 # [1, 2, 3, 1, 2, 3]
```

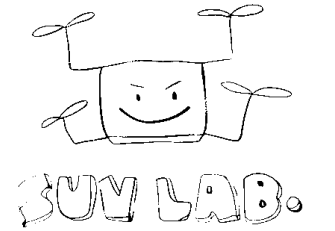
원소마다 파이썬 객체라 메모리가 크고, 계산은 for 루프로 돌아야 한다.

NumPy 배열

```
1 a = np.array([1, 2, 3])
2 a * 2
3 # array([2, 4, 6])
```

같은 타입을 연속된 메모리에 두고, C로 구현된 연산이 배열 전체에 한번에 적용된다.

Pandas, scikit-learn, OpenCV, PyTorch가 모두 NumPy 배열을 기본 자료구조로 쓴다. 데이터 분석의 공통 언어다.

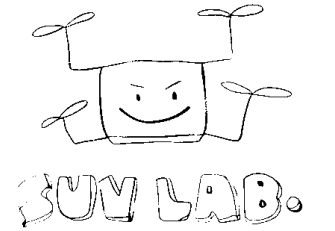


성능 비교 데모

```
1 import numpy as np
2 from time import perf_counter as t
3
4 n = 1_000_000
5 xs = list(range(n))
6 arr = np.arange(n)
7
8 s = t(); sum(xs); py = t() - s
9 s = t(); arr.sum(); nu = t() - s
10 print(f"{py/nu:.0f}배 빠름")
```

- 같은 계산인데 10배 이상 차이가 난다
- 데이터가 커질수록 격차가 벌어진다
- 메모리도 약 4배 이상 절약된다
- 핵심은 **for 루프를 배열 연산으로 바꾸는 것**

숫자는 환경마다 다르다. 중요한 것은 절대값이 아니라 **자릿수가 달라진다는 사실**이다.



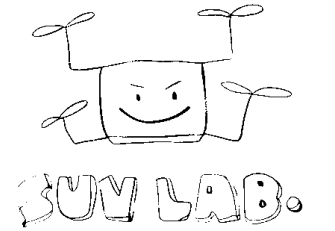
ndarray 기본

같은 타입의 숫자를 **연속된 메모리**에 담은 상자.

- 모든 원소가 같은 dtype을 갖는다
- 크기가 생성 시 정해진다 (append는 새 배열 생성)
- 데이터 본체 + shape 정보로 구성된다
- 1차원은 벡터, 2차원은 행렬, 3차원 이상도 가능

```
1 a = np.array([[1, 2, 3],
2               [4, 5, 6]])
3
4 a.shape      # (2, 3)
5 a.dtype      # int64
6 a.ndim       # 2
```

센서 로그로 치면 **행은 시간, 열은 채널**이 된다. (샘플 수, 3축)



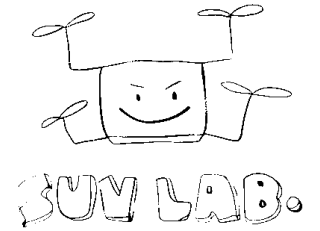
배열 생성

```
1 np.array([1, 2, 3])
2 np.zeros((2, 3))
3 np.ones((2, 3))
4 np.full((2, 2), 7)
5 np.eye(3)           # 단위행렬
6 np.empty((2, 2))   # 초기화 안 함
```

- zeros, ones, full은 shape을 튜플로 받는다
- arange는 간격, linspace는 개수를 지정

```
1 np.arange(0, 10, 2)
2 # [0 2 4 6 8]
3
4 np.linspace(0, 1, 5)
5 # [0.    0.25 0.5   0.75 1.   ]
```

- 길이를 아는 배열은 미리 만들고 채우는 편이 빠르다
- dtype 인자로 타입을 명시할 수 있다



난수 배열

```
1 rng = np.random.default_rng(42)
2
3 rng.random((2, 3))
4 # 0 이상 1 미만 균등
5
6 rng.normal(0, 1, size=100)
7 # 평균 0, 표준편차 1
8
9 rng.integers(1, 7, size=10)
10 # 주사위 10번
```

- 최신 방식은 **default_rng** 제너레이터
- 씨드를 고정해야 결과가 재현된다
- `np.random.seed` 는 구식 API
- 실습용 합성 센서 데이터를 만드는 데 쓴다

논문이나 보고서에 넣을 실험은 **씨드를 반드시 기록**한다. 재현되지 않는 결과는 결과가 아니다.

연습문제 1

균등분포와 정규분포에서 각각 5000개를 뽑아 히스토그램으로 비교하시오. 표본 수와 bins 를 바꾸면 모양이 어떻게 달라지는지 확인하시오.

```
1 import matplotlib.pyplot as plt
2
3 u = rng.uniform(0, 1, 5000)    # 균등분포
4 g = rng.normal(0, 1, 5000)     # 정규분포
5
6 fig, ax = plt.subplots(1, 2, figsize=(9, 3))
7
8 # IMPLEMENT HERE
```


배열 속성

```
1 a = np.zeros((2, 3), dtype=np.float32)
2
3 a.shape      # (2, 3)
4 a.ndim       # 2
5 a.size       # 6
6 a.dtype      # float32
7 a.itemsize   # 4 바이트
8 a.nbytes     # 24 바이트
```

디버깅 첫 줄

배열 코드가 이상하면 먼저 **shape**과 **dtype**을 찍어본다. 오류의 절반은 이 둘 중 하나다.

```
1 print(a.shape, a.dtype)
```

센서 로그처럼 양이 많은 실수 데이터는 **float32**만 써도 충분하다. 메모리가 절반으로 줄어든다.

dtype과 형변환

형변환

```
1 a = np.array([1, 2, 3])
2 a.dtype
3 a.astype(np.float64)
4
5 c = np.array([1.9, 2.9])
6 c.astype(np.int32)
```

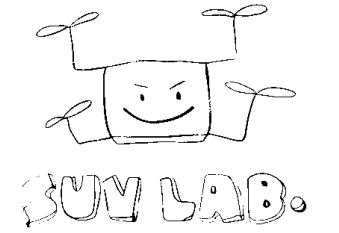
astype 은 항상 새 배열 생성. 제자리 변환 아님

실수에서 정수로는 버림. 반올림이 필요하면 np.round 를 먼저 적용

오버플로우

```
1 x = np.array([127], dtype=np.int8)
2 x.dtype
3 x + 1
```

정수 타입은 표현 범위 고정. 범위를 넘으면 값이 순환
센서 원시값처럼 작은 정수형을 다룰 때 특히 주의



연습문제 2

a 는 $[125, 126, 127]$ 이다. 하지만 $a + 1$ 은 $[126, 127, 128]$ 이 아니다. 원인을 분석하고 코드를 수정하십시오.

```
1 a
2 a + 1
3 np.array_equal(a + 1, [126, 127, 128])
4
5 # IMPLEMENT HERE
```

인덱싱과 슬라이싱

```
1 a = np.arange(12).reshape(3, 4)
2 # [[ 0  1  2  3]
3 #   [ 4  5  6  7]
4 #   [ 8  9 10 11]]
5
6 a[0, 2]      # 2
7 a[1]         # [4 5 6 7]
8 a[:, 1]      # [1 5 9]
9 a[0:2, 1:3]  # [[1 2] [5 6]]
10 a[-1, -1]   # 11
```

- 콤마로 축을 구분한다
- 리스트의 `a[0][2]` 와 달리 한 번에 접근한다
- `:` 는 해당 축 전체를 뜻한다
- 음수 인덱스는 뒤에서부터 센다

뷰(view) 와 복사(copy)

슬라이싱은 뷰다

```
1 a = np.arange(5)
2 b = a[1:4]
3 b[0] = 99
4
5 a # [ 0 99  2  3  4]
```

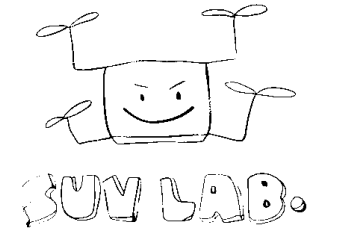
b를 고쳤는데 **a가 변경된다**. 메모리를 공유하기 때문이다.

copy()로 끊는다

```
1 a = np.arange(5)
2 c = a[1:4].copy()
3 c[0] = 99
4
5 a # [0 1 2 3 4]
```

원본을 지켜야 한다면 반드시 **copy()**를 명시한다.

슬라이싱은 **뷰**, 팬시 인덱싱과 불리언 마스킹은 **복사**를 돌려준다. 이 차이가 버그의 단골 원인이다.



연습문제 3

data 는 100x100 배열이다. 원래 그림을 4등분해 사분면 순서를 바꿔 놓았다. 슬라이싱만으로 원래 그림을 복원하시오.

```
1 import matplotlib.pyplot as plt
2
3 fix = data.copy()
4 # IMPLEMENT HERE
5
6 fig, ax = plt.subplots(figsize=(4, 4))
7 ax.imshow(fix, cmap="gray", vmin=0, vmax=1)
```

팬시 인덱싱

```
1 a = np.array([10, 60, 30, 40, 20])
2
3 idx = [4, 0, 2]
4 a[idx]
5
6 order = np.argsort(a)
7 order
8 a[order]
```

- 정수 배열로 원하는 순서 구성
- 같은 인덱스를 여러 번 사용 가능
- 결과는 항상 **새 배열(복사)**
- 2차원은 행 인덱스와 열 인덱스를 함께 전달

```
1 m = np.arange(12).reshape(3, 4)
2 m[[0, 2], [1, 3]]
```

행 인덱스와 열 인덱스를 짝지어 (0, 1) 과 (2, 3) 위치의 값을 뽑는다

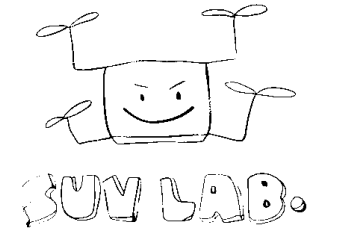
불리언 마스크

```
1 temp = np.array([21.5, 88.0, 22.1, -40.0])
2
3 mask = (temp > -20) & (temp < 60)
4 # [True False True False]
5
6 temp[mask]      # [21.5 22.1]
7 mask.sum()      # 2   조건 만족 개수
8 (~mask).sum()   # 2   이상치 개수
```

and / or 는 안 된다

```
1 temp > -20 and temp < 60
2 # ValueError: truth value
3 # of an array is ambiguous
```

& | ~ 를 쓰고 각 조건을 **괄호**로 묶는다. 연산자 우선순위 때문에 괄호가 필수다.



연습문제 4

temp, hum, place 는 같은 인덱스를 갖는 배열이다. 온도가 30 이상 50 이하이면서 습도가 40 이상인 지점의 이름을 구하시오.

```
1 temp
2 hum
3 place
4
5 # IMPLEMENT HERE
```

where와 조건 처리

```
1 a = np.array([-3, 5, -1, 9])
2
3 np.where(a < 0, 0, a)
4 # [0 5 0 9] 음수를 0으로
5
6 np.clip(a, 0, 6)
7 # [0 5 0 6] 범위 밖을 잘라낸다
8
9 np.where(a < 0)
10 # (array([0, 2]),) 인덱스
```

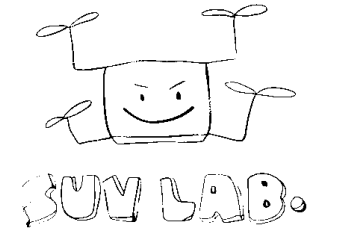
- **where(조건, 참일 때, 거짓일 때)** 세 인자 형태
- 인자가 하나면 조건을 만족하는 인덱스를 돌려준다
- clip은 센서 사양 범위를 강제할 때 편하다
- if 문 대신 쓰는 배열용 조건문이라고 생각하면 된다

형태 변경

```
1 a = np.arange(12)
2
3 a.reshape(3, 4)
4 a.reshape(3, -1)    # -1은 자동 계산
5
6 b = a.reshape(3, 4)
7 b.ravel()           # 1차원 뷰
8 b.flatten()         # 1차원 복사
9 b.T                 # 전치 (4, 3)
```

- reshape는 원소 개수가 맞아야 한다
- -1은 나머지 하나를 알아서 정하라는 뜻
- reshape와 T는 가능하면 뷰를 돌려준다
- ravel은 뷰, flatten은 복사로 기억한다

센서 데이터를 **(샘플, 채널)** 형태로 맞추는 작업이 전처리의 첫 단계다.



연습문제 5

wave 는 0과 1 로 채워진 길이 100 배열이다. 알맞은 형태로 reshape 하면 사인파가 보인다. 그 형태를 찾으시오.

```
1 import matplotlib.pyplot as plt
2
3 img = wave.reshape(10, 10)    # IMPLEMENT HERE
4
5 fig, ax = plt.subplots(figsize=(8, 2))
6 ax.imshow(img, cmap="gray_r", vmin=0, vmax=1)
```

축 추가와 제거

```
1 a = np.array([1, 2, 3])    # (3,)
2
3 a[:, np.newaxis]          # (3, 1) 열벡터
4 a[np.newaxis, :]          # (1, 3) 행벡터
5
6 np.expand_dims(a, axis=0)
7 # (1, 3)
8
9 np.squeeze(np.zeros((1, 3, 1)))
10 # (3,) 크기 1인 축 제거
```

- 브로드캐스팅 모양을 맞추려고 축을 늘린다
- None 은 np.newaxis 와 같다
- squeeze는 쓸모없는 1짜리 축을 걸러낸다
- shape을 입단으로 말해 보면 실수가 줄어든다

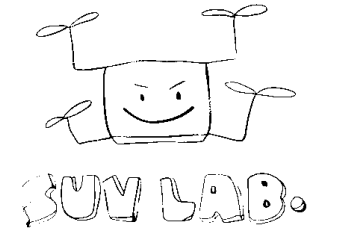
배열 결합과 분할

```
1 a = np.array([[1, 2]])
2 b = np.array([[3, 4]])
3
4 np.vstack([a, b]) # (2, 2) 세로
5 np.hstack([a, b]) # (1, 4) 가로
6
7 np.concatenate([a, b], axis=0)
8 # 축을 명시하는 일반형
```

```
1 x = np.arange(6)
2
3 np.split(x, 3)
4 # [array([0,1]), array([2,3]),
5 #   array([4,5])]
6
7 np.array_split(x, 4)
8 # 나누어 떨어져도 된다
```

결합(vstack, hstack, concatenate)은 **새 메모리를 할당**한다.

루프 안에서 반복해 붙이면 느리다. 리스트에 모은 뒤 마지막에 한 번만 concatenate 한다.



요소별 연산과 벡터화

```
1 a = np.array([1, 2, 3])
2 b = np.array([10, 20, 30])
3
4 a + b      # [11 22 33]
5 a * b      # [10 40 90]
6 b / a      # [10. 10. 10.]
7 a ** 2     # [1 4 9]
```

$$\begin{array}{|c|c|c|} \hline a(3,) \\ \hline 1 & 2 & 3 \\ \hline \end{array} \times \begin{array}{|c|c|c|} \hline 2 \text{ scalar} \\ \hline 2 & 2 & 2 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline (3,) \\ \hline 2 & 4 & 6 \\ \hline \end{array}$$

스칼라 2가 배열 크기만큼 늘어나 곱해진다.

벡터화: 루프 없이 배열 전체에 한번에 적용한다. sqrt, exp, log, sin 같은 **유니버설 함수(ufunc)**도 마찬가지다.

브로드캐스팅 규칙

- 1 맨 뒤쪽 축부터 하나씩 비교한다
- 2 크기가 같거나 한쪽이 1이면 통과
- 3 1인 축을 복사하듯이 늘려 맞춘다
- 4 그 외에는 오류를 낸다

```

1 (4, 3) + (3,)      # OK
2 (4, 3) + (4, 1)    # OK
3 (4, 3) + (4,)      # ValueError

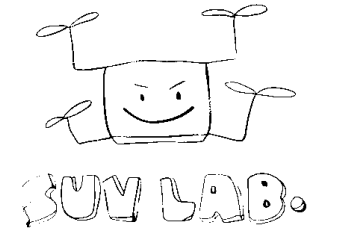
```

(4, 3)		(3,)		(4, 3)
1 2 3		10 20 30		11 22 33
1 2 3	+	10 20 30	=	11 22 33
1 2 3		10 20 30		11 22 33
1 2 3		10 20 30		11 22 33

(4,3) + (3,) — 뒤쪽 축이 맞아 (3,)이 행마다 복사되듯 늘어난다

(4, 3)		(4,)
1 2 3		1 2 3 4
1 2 3	+	3 != 4
1 2 3		ValueError
1 2 3		

(4,3) + (4,) — 뒤쪽 축 3과 4가 달라 오류.



연습문제 6

a 는 (3,), b 는 (4,) 이다. 지금은 오류가 난다. reshape 만 사용해 $a + b$ 가 (3, 4) 결과를 내도록 고치시오.

$$a \in \mathbb{R}^3 \quad b \in \mathbb{R}^4 \quad a + b \in \mathbb{R}^{3 \times 4}$$

```
1 a.shape
2 b.shape
3 a + b
4
5 # IMPLEMENT HERE
```

브로드캐스팅 활용

```

1 data = rng.normal(size=(100, 3))
2
3 col_mean = data.mean(axis=0) # (3,)
4 col_std  = data.std(axis=0)  # (3,)
5
6 centered = data - col_mean
7 normed   = (data - col_mean) / col_std
8 # 둘 다 (100, 3)

```

```

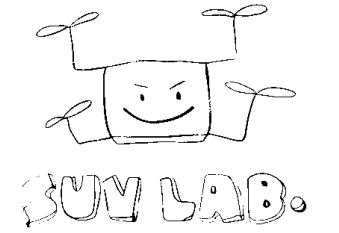
1 bias = np.array([0.02, -0.01, 0.05])
2 calibrated = data - bias # 축별 영점 보정

```

$$\begin{array}{|c|c|c|} \hline (4, 1) & & (3,) \\ \hline 0 & 0 & 0 \\ \hline 10 & 10 & 10 \\ \hline 20 & 20 & 20 \\ \hline 30 & 30 & 30 \\ \hline \end{array} + \begin{array}{|c|c|c|} \hline (3,) & & \\ \hline 1 & 2 & 3 \\ \hline 1 & 2 & 3 \\ \hline 1 & 2 & 3 \\ \hline 1 & 2 & 3 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline (4, 3) & & \\ \hline 1 & 2 & 3 \\ \hline 11 & 12 & 13 \\ \hline 21 & 22 & 23 \\ \hline 31 & 32 & 33 \\ \hline \end{array}$$

(4,1) + (3,) — 양쪽이 다 늘어나 입력보다 큰 (4,3)이 된다.

- 열별 평균 빼기와 표준화가 한 줄이다
- 센서 채널별 보정값 적용도 같은 패턴을 쓴다



연습문제 7

u 와 v 의 곱을 먼저 손으로 예측한 뒤 실행해 확인하시오.

$$u \in \mathbb{R}^{3 \times 1} \quad v \in \mathbb{R}^4 \quad (u \odot v)_{ij} = u_i v_j$$

```
1 u
2 v
3 u * v
4
5 # IMPLEMENT HERE
```

집계 함수

```
1 a = np.array([[1, 2, 3],
2               [4, 5, 6]])
3
4 a.sum()      # 21
5 a.mean()     # 3.5
6 a.min()      # 1
7 a.max()      # 6
8 a.std()      # 1.707...
9 a.argmax()   # 5   평탄화 기준 위치
```

sum

합계

std, var

표준편차, 분산

argmin, argmax

최솟값 위치

mean

평균

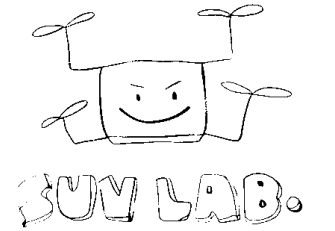
median

중앙값

cumsum

누적합

argmax는 값이 아니라 위치를 돌려준다. 충격이 가장 컸 순간을 찾을 때 쓴다.



axis 이해하기

axis는 사라지는 축이다.

```
1 a = np.array([[1, 2, 3],
2               [4, 5, 6]]) # (2, 3)
3
4 a.sum(axis=0) # [5 7 9] shape (3,)
5 a.sum(axis=1) # [6 15] shape (2,)
6
7 a.sum(axis=0, keepdims=True)
8 # [[5 7 9]] shape (1, 3)
```

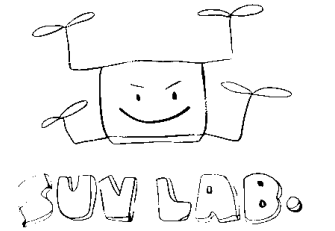
- axis=0은 행을 따라 접어 **열별 결과**
- axis=1은 열을 따라 접어 **행별 결과**
- keepdims=True는 축을 1로 남겨 브로드캐스팅을 쉽게 한다
- 센서 로그 (샘플, 채널)에서 axis=0은 시간 방향 집계다

결측값(nan) 처리

```
1 a = np.array([1.0, np.nan, 3.0])
2
3 a.mean()          # nan   오염된다
4 np.nanmean(a)     # 2.0
5
6 np.isnan(a)       # [False True False]
7 a[np.isnan(a)] = 0.0
8
9 np.nan == np.nan  # False
```

nan의 성질

- nan은 float 배열에만 존재한다
- 비교 연산으로 찾을 수 없다
- 하나만 있어도 전체 통계가 nan이 된다
- nansum, nanmean, nanstd, nanmax 를 쓴다



정렬과 탐색

```
1 a = np.array([3, 1, 2])
2
3 np.sort(a)      # [1 2 3] 복사본
4 a.sort()        # 제자리 정렬
5 np.argsort(a)   # 정렬 순서 인덱스
```

- **np.sort**는 복사본, **a.sort()**는 원본을 바꿈
- 2차원에서는 axis를 지정해 행별/열별 정렬

```
1 np.unique([1, 1, 2]) # [1 2]
2
3 np.searchsorted([1, 3, 5], 4)
4 # 2 정렬 상태를 유지할 위치
```

- argsort로 다른 배열을 같은 순서로 재배열한다
- 시간순 정렬이 깨진 로그를 바로잡을 때 쓴다

행렬 연산

$$(A \odot A)_{ij} = A_{ij}A_{ij} \quad (AA)_{ij} = \sum_k A_{ik}A_{kj} \quad (A^T)_{ij} = A_{ji}$$

* 는 요소별 곱

```
1 A = np.array([[1, 2],  
2               [3, 4]])  
3 A * A  
4 # [[ 1  4]  
5 #  [ 9 16]]
```

@ 는 행렬 곱

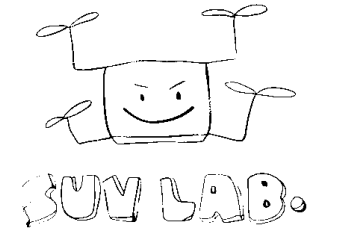
```
1 A @ A  
2 # [[ 7 10]  
3 #  [15 22]]  
4  
5 A.T      # 전치  
6 A.dot(A) # @ 와 같다
```

수식을 그대로 옮길 때 **곱셈 기호를 확인**하는 습관이 필요하다. 둘은 shape은 같아도 결과가 전혀 다르다.

선형대수 모듈

```
1 from numpy.linalg import inv, solve, det, norm
2
3 A = np.array([[2., 1.],
4               [1., 3.]])
5 b = np.array([5., 10.])
6
7 solve(A, b)    # Ax = b 의 해
8 inv(A)         # 역행렬
9 det(A)         # 행렬식
10 norm(b)        # 벡터의 크기
```

- **inv(A) @ b** 보다 **solve(A, b)**가 빠르고 안정적이다
- **eig, svd, qr** 등 분해도 제공된다
- 최소자승법은 **lstsq**로 한 줄에 풀 수 있다
- 칼만 필터, 자세 추정의 기본 도구다

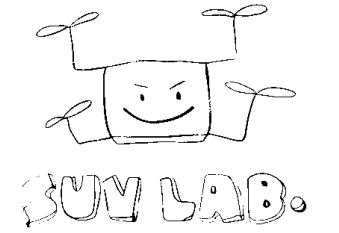


연습문제 8

$Ax = b$ 를 푸는 x 를 구하시오. 이어서 $A * A$ 와 $A @ A$ 의 결과가 어떻게 다른지 확인하시오.

$$Ax = b \quad x = A^{-1}b \quad (A \odot A)_{ij} = A_{ij}A_{ij} \quad (AA)_{ij} = \sum_k A_{ik}A_{kj}$$

```
1 A
2 b
3
4 # IMPLEMENT HERE
```

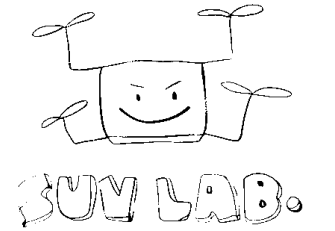


연습문제 9

pts 는 점을 행으로 쌓은 (N, 2) 배열이다. $t = 30$ 도 회전 행렬 $R(t)$ 를 만들어 pts 를 회전 시키시오.

$$R(t) = \begin{bmatrix} \cos t & -\sin t \\ \sin t & \cos t \end{bmatrix} \quad P' = PR^T$$

```
1 t = np.deg2rad(30)
2 pts
3
4 # IMPLEMENT HERE
```



파일 입출력

```
1 np.save("data.npy", a)
2 b = np.load("data.npy")
3
4 np.savez("log.npz", imu=a, gps=b)
5 d = np.load("log.npz")
6 d["imu"]
```

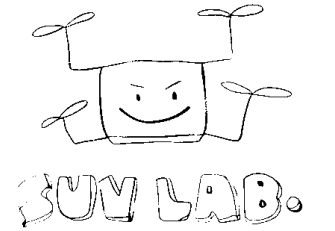
npz / npy

이진 형식. 빠르고 **dtype**과 **shape**이 그대로 보존된다. 중간 결과 저장에 적합.

```
1 np.savetxt("data.csv", a,
2           delimiter=",")
3
4 np.loadtxt("data.csv",
5           delimiter=",")
```

csv / txt

사람이 읽기 좋고 다른 도구와 주고받기 쉽다. 대신 느리고 정밀도 손실이 생길 수 있다.



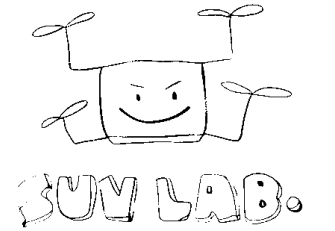
자주 하는 실수

이런 일이 생긴다

- 슬라이스가 뷰인 줄 모르고 원본을 망가뜨린다
- and / or 를 써서 ValueError가 난다
- shape이 맞지 않아 브로드캐스팅 오류가 난다
- axis를 반대로 줘 엉뚱한 통계가 나온다
- 정수 dtype 오버플로우를 못 보고 지나간다

이렇게 막는다

- 의심되면 shape과 dtype을 먼저 출력한다
- 원본을 지켜야 하면 copy()를 명시한다
- 조건은 & | ~ 와 괄호로 쓴다
- axis는 결과 shape으로 검사한다
- 센서 원시값은 읽자마자 float로 바꿔둔다



실습 안내 (lab03)

- 1 합성 가속도 3축 배열 생성 (씨드 고정)
- 2 물리 범위 밖 값을 마스킹으로 걸러내기
- 3 축별 평균, 표준편차, 최대 가속도 시점
- 4 5샘플 이동평균로 스무딩
- 5 회전 행렬로 센서 좌표계 정렬

체크포인트

- for 루프 없이 작성했는가
- 원본 배열이 그대로 남아 있는가
- 결과 shape을 설명할 수 있는가

30분

[lab03-numpy.md](#)

다음: **Part 2-3 Pandas** — 라벨이 붙은 표 형태 데이터와 전처리.